COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S4
DATE PERFORMED:	08/26/2026	**DATE SUBMITTED:**	08/26/26
NAME:	DEL MUNDO RAMIREZ MANGCUCANG FORMALEJO LICAS LACSAMANA	**INSTRUCTOR:**	**Engr. Jimlord Q. Quejado**

PRELIM EXAM **PRELIM LABORATORY EXAM**

1. PROCEDURE OUTPUT

Problem Title: The Customer Service Verification Problem

A customer service center processes customers in the order they arrive. Each customer is assigned a unique customer number.

At the end of the day, the supervisor wants to verify whether the order in which the customers were served is the **reverse of the order in which they originally arrived**.

Given a sequence of customer numbers, your task is to determine whether the sequence is the same when its order is reversed.

For example:

101 102 103 102 101

is a valid reverse-order sequence because the values read from the beginning and from the end are the same.

Your task is to use **two data structures** for your solution:

- Stack
- Queue

However, the two data structures **must have different internal representations**, either:

- Array-based
- Linked List

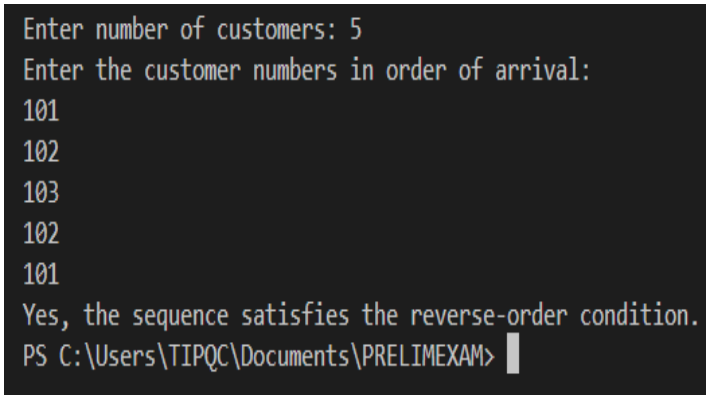
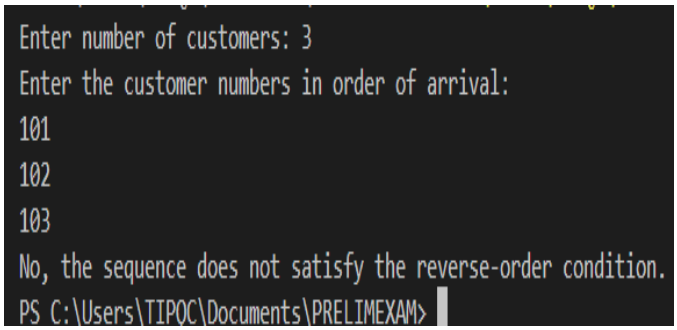
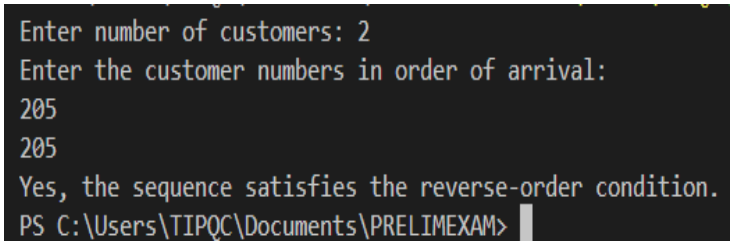
Example: If your stack is array-based, your queue must be implemented using a linked list, and vice versa.

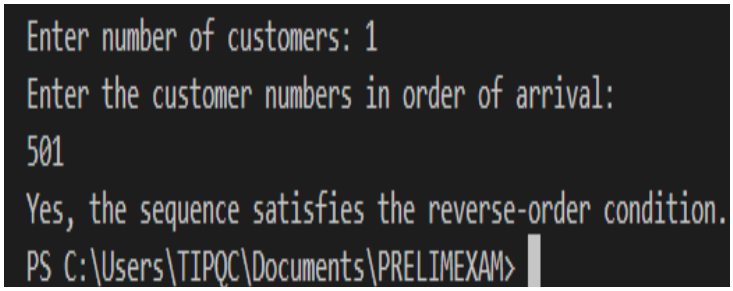
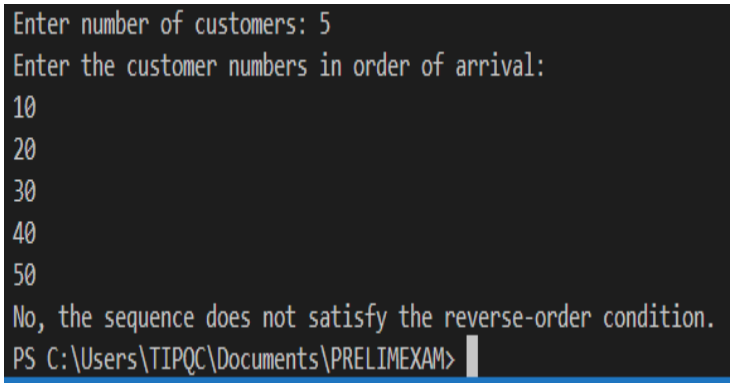
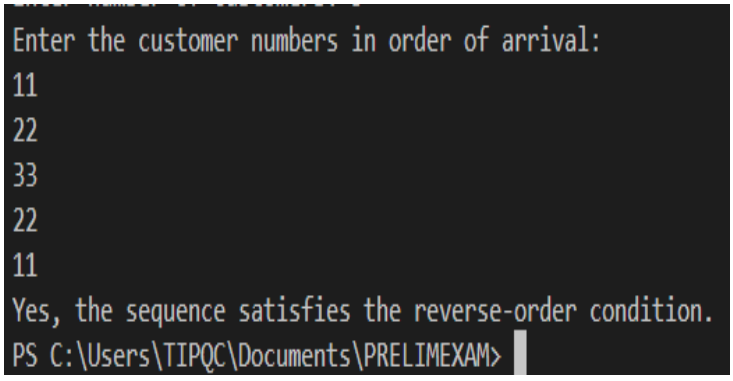
Each customer number must be inserted into **both the Stack and Queue**.

You must then use the behavior of the two data structures to determine whether the sequence satisfies the required condition.

You may only use the **public operations provided by the existing Stack and Queue header files**. Do not modify the internal representation of either data structure.

The user is then prompted whether the given customer sequence satisfies the reverse-order condition.

Customer Numbers	Valid?	Output (Console)	Remark
101 102 103 102 101	Yes		Yes, the sequence satisfies the reverse-order condition.
101 102 103	No		No, the sequence does not satisfy the reverse-order condition.
205 205	Yes		Yes, the sequence satisfies the reverse-order condition.

501	Yes		Yes, the sequence satisfies the reverse-order condition.
10 20 30 40 50	No		No, the sequence does not satisfy the reverse-order condition.
11 22 33 22 11	Yes		Yes, the sequence satisfies the reverse-order condition.

MAIN.CPP

main.cpp > main()

```cpp
#include <iostream>
#include "Stack.h"
#include "Queue.h"
using namespace std;

bool isReverseOrder(Stack& customerStack, Queue& customerQueue, int n) {
    bool isValid = true;
    for (int i = 0; i < n; i++) {
        int fromStack = customerStack.pop();
        int fromQueue = customerQueue.dequeue();
        if (fromStack != fromQueue) {
            isValid = false;
        }
    } <- #8-14 for (int i = 0; i < n; i++)
    return isValid;
} <- #6-16 bool isReverseOrder(Stack& customerStack, Queue& customerQueu...

int main() {
    int n;
    cout << "Enter number of customers: ";
    cin >> n;

    Stack customerStack;
    Queue customerQueue;

    cout << "Enter the customer numbers in order of arrival:\n";
    for (int i = 0; i < n; i++) {
        int num;
        cin >> num;
        customerStack.push(num);
        customerQueue.enqueue(num);
    } <- #27-32 for (int i = 0; i < n; i++)

    bool result = isReverseOrder(customerStack, customerQueue, n);

    if (result) {
        cout << "Yes, the sequence satisfies the reverse-order condition.\n";
    } else {
```

```
35
36      if (result) {
37          cout << "Yes, the sequence satisfies the reverse-order condition.\n";
38      } else {
39          cout << "No, the sequence does not satisfy the reverse-order condition.\n";
40      }
41
42      return 0;
43  } <- #18-43 int main()
```

QUEUE.H

Queue.h > ...

```
1
2   #ifndef QUEUE_H
3   #define QUEUE_H
4
5   #include <iostream>
6   using namespace std;
7
8   struct QueueNode {
9       int data;
10      QueueNode* next;
11  };
12
13  class Queue {
14  private:
15      QueueNode* frontPtr;
16      QueueNode* backPtr;
17
18  public:
19      Queue() {
20          frontPtr = nullptr;
21          backPtr = nullptr;
22      }
23
24      ~Queue() {
25          while (frontPtr != nullptr) {
26              QueueNode* temp = frontPtr;
27              frontPtr = frontPtr->next;
28              delete temp;
29          } <- #25-29 while (frontPtr != nullptr)
30      } <- #24-30 ~Queue()
31
32      bool isEmpty() const {
33          return frontPtr == nullptr;
34      }
35
36      void enqueue(int val) {
37          QueueNode* newNode = new QueueNode;
```

```
        newNode->next = nullptr;
        if (isEmpty()) {
            frontPtr = newNode;
            backPtr = newNode;
        } else {
            backPtr->next = newNode;
            backPtr = newNode;
        }
    } <- #36-47 void enqueue(int val)

    int dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty." << endl;
            return -1;
        }
        QueueNode* temp = frontPtr;
        int val = temp->data;
        frontPtr = frontPtr->next;
        if (frontPtr == nullptr) {
            backPtr = nullptr;
        }
        delete temp;
        return val;
    } <- #49-62 int dequeue()

    int front() const {
        if (isEmpty()) {
            cout << "Queue is empty." << endl;
            return -1;
        }
        return frontPtr->data;
    } <- #64-70 int front() const
}; <- #13-71 class Queue

#endif
```

STACK.H

Stack.h > Stack

```h
#ifndef STACK_H
#define STACK_H

#include <iostream>
using namespace std;

class Stack {
private:
    int arr[100];
    int topIndex;

public:
    Stack() {
        topIndex = -1;
    }

    bool isEmpty() const {
        return topIndex == -1;
    }

    bool isFull() const {
        return topIndex == 99;
    }

    void push(int val) {
        if (isFull()) {
            cout << "Stack is full." << endl;
            return;
        }
        topIndex++;
        arr[topIndex] = val;
    } <- #25-32 void push(int val)

    int pop() {
        if (isEmpty()) {
            cout << "Stack is empty." << endl;
```

```
37              return -1;
38          }
39          int val = arr[topIndex];
40          topIndex--;
41          return val;
42      } <- #34-42 int pop()
43
44      int peek() const {
45          if (isEmpty()) {
46              cout << "Stack is empty." << endl;
47              return -1;
48          }
49          return arr[topIndex];
50      } <- #44-50 int peek() const
51  }; <- #7-51 class Stack
52
53  #endif
54
```

ANALYSIS

In this activity, we checked if the customer numbers match the reverse order by storing each customer number in a stack and a queue. We use the queue to keep the original first come first serve order. Then, we used the stack to flip the order around. In our reverseorder function, we loop all the customer numbers then we pull one value from the stack and another one from the queue using. Then it will compare the values. If all single pairs match, the sequence is symmetric and reads it all. If there is one pair that does not match, we set isvalid to false because the sequence will fail the reverseorder check.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.